

Blueprint Técnico — Registro de Unidades Agênticas (RUA)

v1.0 · 2026-06-03 · referência de engenharia do artefato

Companheiro técnico do `MC-MANUAL-Registro-Unidades-Agenticas` (que é Founder-facing). Aqui o nível é de **implementação**: estruturas de dados, fluxo de funções, o interior do painel, garantias de determinismo, o firewall LGPD e os pontos de extensão. Tudo aferido **contra o código real** em `.claude/registro/` e `.claude/hooks/` — Proof-First, zero fabricação.

1 · Visão & escopo

O **RUA** é o **plano de controle** sobre o plano de dados agêntico do MC. Os agentes executam (skills, agents, hooks, workflows em `.claude/`); o RUA sabe **quem são, em que gate cada um para, que zona LGPD toca, quanto custa e o que não confundir com o quê**.

É a **camada de instrumentação** da stack **ASE (Agentic Systems Engineering)**. Mapa de camadas e o que foi implementado:

Camada	Papel	Implementado?
L0	Artefatos crus (<code>.claude/skills/*</code> , <code>agents/*</code> , <code>hooks/*</code> , <code>workflows/*</code>)	pré-existente
L1	Registro — projeção determinística dos artefatos	<code>gerar-registro.py</code>
L2-L4	Execução / roteamento / enforcement (<code>mc-lint</code> vive aqui)	parcial (fora do escopo RUA)
L5	Telemetria — série temporal de execuções + alça de otimização	<code>mc-audit.py</code> + <code>ler-audit.py</code>
L6	Cockpit — visualização autossuficiente	<code>gerar-cockpit.py</code> → <code>painel-mc.html</code>

Cobertura atual: **18 Unidades Agênticas (UAs) · 100% instrumentadas** (14 skill · 2 hook · 2 agent).

2 · Princípios de design (invioláveis no código)

- Gerado, não mantido.** O Registro **nunca** é editado à mão. A fonte da verdade são os artefatos (o bloco `rua:` no frontmatter de cada UA). O `.json/ .md/ .html` são colheita. Editar a saída à mão = drift = passivo (viola Proof-First).
- Determinístico.** Rodar o gerador 2× no mesmo commit → **saída byte-idêntica**. Sem `Date.now()/ random`: o carimbo de origem é o **SHA do commit** (`git rev-parse --short HEAD`). (Exceção legítima: a telemetria L5 é série temporal — carrega wall-clock por natureza; o determinismo lá é do analisador sobre um log fixo, não do log.)
- Fail-closed.** UA sem um campo `★` obrigatório **não entra** no Registro canônico — vai para `incompletas`, visível como dívida, nunca silenciada.
- Zona Verde · zero PII.** O Registro só carrega metadado de governança. O cockpit (L6) é compartilhável ⇒ tratado como público ⇒ nenhum dado de caso entra.
- Repo-relativo.** Todo path resolve a partir do próprio script (`__file__`), nunca de `cwd` ou caminho absoluto — roda igual em sessão local e na nuvem.

6. **Para no gate humano.** Nenhum componente sela, injeta ou escreve no Vault. Devolve artefatos derivados na drop-zone e para.

3 · Arquitetura de dados

3.1 A ontologia — o bloco `rua:` (5 blocos · MC-SPEC §3)

Cada UA declara, no próprio frontmatter (ou num sidecar `.rua.yml` para hooks, que não têm frontmatter), um bloco `rua:`. Campos ★ são obrigatórios (fail-closed):

Bloco	Campos	★ obrigatórios
A · Identidade	<code>id</code> , <code>nome</code> , <code>tipo</code> , <code>versao</code> , <code>status</code> , <code>data</code>	<code>id</code> , <code>nome</code> , <code>tipo</code> , <code>versao</code> , <code>status</code>
B · Função	<code>papel</code> , <code>gatilhos</code> , <code>entrega</code> , <code>metodo_fonte</code>	<code>todos</code>
C · Posição MC-ACI	<code>instancia_aci</code> , <code>camada_ase</code> , <code>compoe_com</code> , <code>nao_confundir_com</code>	<code>instancia_aci</code> , <code>nao_confundir_com</code>
D · Governança	<code>gate</code> , <code>zona</code> , <code>lgpd_notas</code> , <code>vedacoes_honradas</code> , <code>para_no_gate</code>	<code>gate</code> , <code>zona</code> , <code>vedacoes_honradas</code> , <code>para_no_gate</code>
E · Operação	<code>custo</code> , <code>regressao</code> , <code>emite_audit</code> , <code>depende_de</code>	<code>custo</code>

REQUIRED (16 campos) é a régua fail-closed do gerador. `nao_confundir_com` pode ser lista vazia (um "explicitamente nada" é válido); os demais ★ não.

3.2 O schema da linha de auditoria (conjunto FECHADO — o firewall em forma de dados)

Uma linha do `audit.jsonl` só pode conter estas 9 chaves. Nenhuma outra é jamais escrita:

```
ts · event · ua · tipo · gate · zona · instancia_aci · in_registro · sessao_hash
```

Esta lista É o firewall LGPD: nada do payload de invocação (args, prompt, anexos, caminhos) tem porta de entrada. Detalhe em §4.3.

4 · Os componentes em profundidade

4.1 L1 · `gerar-registro.py` — o recenseador

Entrada: varredura ordenada (determinística) de `.claude/` - `skills/*/SKILL.md` → `read_rua()` extrai só o bloco `rua:` do frontmatter (parse robusto: fatia de `^rua:` até o fim e dá `yaml.safe_load` só nisso — descrições legadas com `:` em scalar plano quebrariam YAML estrito do frontmatter inteiro). - `agents/*.md` → idem. - `hooks/*.py` → procura sidecar `<nome>.rua.yml` (hook `.py` não tem frontmatter). - `workflows/*.js` → lê `meta.name`; **dobra** o workflow na UA dona (`id == name` ou `name.startswith(id+"-")`). Workflow sem dona → dívida.

Classificação: `_add()` roda `missing_fields()`. Completa → `unidades` (+ `ids`). Incompleta → `incompletas` com `campos_faltantes`. Sem bloco → `nao_instrumentadas`.

Determinismo: `commit_sha()` carimba `meta.gerado_de_commit`; listas ordenadas por `id`; `json.dump(..., sort_keys=False, ensure_ascii=False, indent=2)`.

Saídas: `registro-unidades-agenticas.json` (máquina) + `REGISTRO-Unidades-Agenticas.md` (view humana com tabela de cobertura).

4.2 L6 · `gerar-cockpit.py` + `painel-mc.html` — o cockpit (o painel que construímos)

Esta é a peça que o Founder mais usa, então vai em detalhe.

Modelo "dashboard autossuficiente" (molde f2810ce). O gerador lê o `.json`, serializa os dados e os embute dentro de um `<script>` de um único HTML. Resultado: **1 arquivo, zero servidor, zero dependência externa, zero rede.** Abre em qualquer navegador ou celular, cola no Claude, manda no zap.

Endurecimento fail-closed do embed (defesa em profundidade):

```
DATA_JSON = json.dumps(REG, ensure_ascii=False).replace("<", "\\u003c")
```

Escapar `<` como `<` mantém JSON válido e garante que **nenhum dado futuro** (ex.: um gatilho que algum dia contenha `</script>`) consiga quebrar o `<script>` embutido. É o mesmo espírito do firewall: o dado não tem como escapar do contêiner.

Pipeline: `TEMPLATE.replace("__DATA__", DATA_JSON)` → grava `painel-mc.html`. Determinístico (sem wall-clock; o `meta.gerado_de_commit` vem do `.json`).

Anatomia do `painel-mc.html` (lado cliente, JS puro):

Região	Como funciona	...
Tema	CSS vars on-brand MC: fundo <code>#120d08</code> , ouro <code>#d4af37</code> , zonas Verde/Amarela/Vermelha e gates R1/R2/R3 com cores próprias.	
Header + meta	commit de origem · nº de UAs · % cobertura · "gerado, não mantido".	
Stats	contadores derivados em runtime: total · R1/R2/R3 · / / (<code>count(field,val)</code>).	
Busca	<code>#q</code> filtra por <code>id + nome + papel + gatilhos</code> (lowercase, substring).	
Chips	filtros multi-seleção gerados de <code>uniq("tipo" "gate" "zona" "status")</code> . Conjuntos <code>FILTERS</code> ; clique alterna.	
Cards	um por UA. Badges de <code>tipo</code> , <code>gate</code> (mapa <code>RB</code>), <code>zona</code> (mapa <code>ZB</code>); UAs <code>SUPERSEDED</code> esmaecidas (<code>.card.sup</code>).	
POP	expande no clique (<code>.card.open .pop</code>): Quando usar · Entrega · Para no gate · Custo · LGPD · Vedações honradas · Não confundir com · Método-fonte · Instância MC-ACI/camada ASE · Artefatos.	
Footer	"Mostrando N/total" + selo <code>D>C>V</code> .	
Segurança	<code>esc()</code> escapa <code>& < ></code> em todo campo renderizado (defesa XSS no cliente, além do escape do embed).	

Disciplina de governança no cockpit (MC-SPEC §4.2): o painel **destaca** UAs R2/R3 e **nunca** oferece "selar" como botão. Selo é julgamento humano soberano; a ferramenta rastreia, não decide.

4.3 L5 · `mc-audit.py` — o emissor (hook)

Protocolo: hook do Claude Code; recebe JSON do evento em `stdin` (`PreToolUse`/`PostToolUse` sobre Skill/Task/Agent).

O firewall, passo a passo: 1. `_resolver_ua(payload)` lê **uma** chave whitelisted — `tool_input.skill` ou `tool_input.subagent_type` — e devolve só o `id`. **Todo o resto do `tool_input` é ignorado.** Não há caminho de código que copie o payload. 2. `_carregar_registro()` indexa o `.json` por `id` (une `unidades` + `incompletas`). 3. A linha é montada **só** com o conjunto fechado (§3.2); metadados (`tipo/gate/zona/ instancia_aci`) vêm do **Registro**, não do payload. 4. `_hash_sessao() = sha256(session_id)[:8]` — conta sessões distintas **sem reter o id**. 5. Cinto de segurança final: `linha = {k: linha[k] for k in CHAVES_PERMITIDAS}` — descarta qualquer chave fora do conjunto fechado, por bug futuro que seja. 6. Apenas 1 linha em `audit/audit.jsonl`.

Consequência verificada: payload com `novo-cliente` + CPF/NB/CID nos args → log grava só "novo-cliente rodou em T · R1 · Vermelha". **Zero vazamento.** Mesmo UA Vermelha produz só metadado de governança ⇒ o log é Zona Verde.

Políticas: **fail-safe** (qualquer erro → `exit 0`; telemetria nunca bloqueia o turno, nunca derruba a sessão). **DEFAULT OFF** — não está fiado em `settings.json`; ligar a captura ao vivo é o **ato R2** do Founder (#50.2: o agente não se autoinstala).

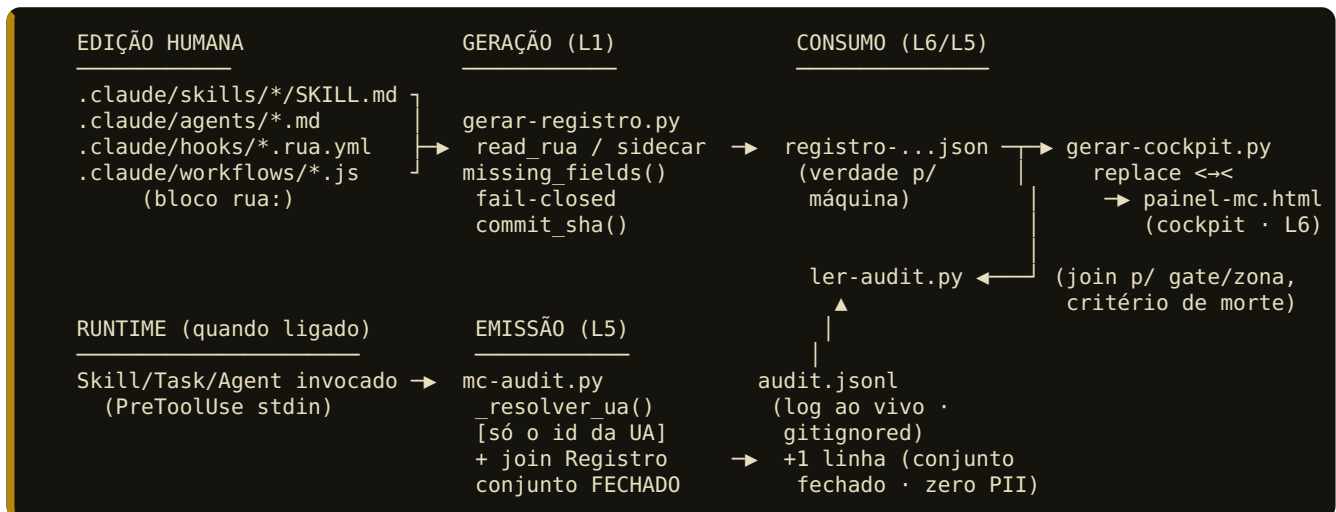
4.4 L5 · `ler-audit.py` — a alça de otimização

Entrada: `audit.jsonl` ao vivo, ou `--fixture` (golden sintético), ou `--in PATH`. Linhas-sentinela (`event` começa com `_`) são descartadas antes da métrica (Proof-First: o rótulo "sintético" vive no dado sem poluir a contagem).

O que computa (a "alça"): - **Ranking** de invocações por UA (count desc, id asc). - **Critério de morte** (MC-SPEC §6): UAs do Registro que **nunca** apareceram no log. - **Carga de governança:** invocações por gate (R1/R2/R3) — quanto de supervisão humana o tráfego consome. (Classificação pela verdade atual do Registro, não pelo carimbo do log.) - **Superfície LGPD:** invocações por zona (Verde/Amarela/Vermelha). - **Dívida de cobertura em runtime:** `in_registro:false` (rodou sem bloco `rua:`). - **Período coberto:** min/max `ts` dos dados (determinístico, não wall-clock).

Determinismo: mesmo commit + mesmo JSONL → relatório byte-idêntico (verificado por hash).

5 · Fluxo de dados (function-level)



6 · Determinismo & regressão

- **Geradores L1/L6:** prova = rodar 2x no mesmo commit e comparar SHA-256 da saída (byte-idêntico). O único delta legítimo entre commits é `meta.gerado_de_commit`.
- **Analizador L5:** golden fixture `audit/audit-FIXTURE.jsonl` (40 linhas: 1 sentinela Proof-First + 39 invocações **sintéticas rotuladas** · 11 UAs rodando · 6 nunca) → `RESUMO-Telemetria-FIXTURE.md` é o golden de regressão. Mesmas entradas → saída idêntica.
- **Padrão de fixtures** herdado do ecossistema MC (golden master Hib001 do motor de espécimen): "unit tests for English".

7 · Governança embarcada

- **Gates (R1/R2/R3):** dado de primeira classe. Hoje: 14 R1 · 1 R2 (`mc-audit`) · 3 R3.
- **Zonas (Verde/Amarela/Vermelha):** 14 · 2 · 2.

- **Vedações honradas** por UA: `D>C>V`, `Firewall-0AB`, `Proof-First`, `LGPD`, `Zona-Verde`, `#50.2` — contrato ético explícito e auditável.
- **#50.2 Cabeça Única**: o RUA é cockpit do Founder, não terminal multiusuário; nenhum componente é autônomo.
- **Catraca R3**: a Fase 4 (telemetria) escala para R3 (Dra. Juliana) **apenas se** o conjunto fechado de chaves passar a tocar fluxo PII entre instâncias C4/C5 — o que o desenho proíbe. Por isso, hoje, **R2 basta** para ligar a captura.

8 · Pontos de extensão / roadmap

- **Ligar a telemetria (ato R2)**: bloco `PreToolUse` matcher `Skill|Task` em `.claude/settings.json` chamando `mc-audit.py` (instrução exata em `audit/_LEIA-audit.md`).
- **Fase 4+ (R2/R3)**: elevar o emissor para **spans OpenTelemetry GenAI** (`invoke_agent` / `invoke_workflow`) — padrão convergente de observabilidade agêntica.
- **Novas UAs**: nascem instrumentadas (bloco `rua:`); regenerar Registro → cockpit. A cobertura e o critério de morte se atualizam sozinhos.

9 · Inventário de arquivos do artefato

Arquivo	Camada	Tipo
<code>.claude/registro/gerar-registro.py</code>	L1	gerador
<code>.claude/registro/registro-unidades-agenticas.json</code>	L1	gerado (máquina)
<code>.claude/registro/REGISTRO-Unidades-Agenticas.md</code>	L1	gerado (view)
<code>.claude/registro/gerar-cockpit.py</code>	L6	gerador
<code>.claude/registro/painel-mc.html</code>	L6	gerado (cockpit)
<code>.claude/hooks/mc-audit.py</code> (+ <code>.rua.yml</code>)	L5	emissor (hook)
<code>.claude/registro/ler-audit.py</code>	L5	analizador
<code>.claude/registro/audit/audit-FIXTURE.jsonl</code>	L5	fixture sintético (golden)
<code>.claude/registro/audit/RESUMO-Telemetria-FIXTURE.md</code>	L5	gerado (golden)
<code>.claude/registro/audit/.gitignore</code>	L5	ignora o log ao vivo (efêmero)
<code>.claude/registro/_LEIA-registro.md</code> · <code>audit/_LEIA-audit.md</code>	—	docs operacionais
<code>02-ARQUITETURA/specs/MC-SPEC-RegistroAgentes-ASE-v0_1-...md</code>	—	ontologia (gate R2)
<code>02-ARQUITETURA/manuais/MC-MANUAL-...pdf</code>	—	manual Founder-facing

`mc-lint.py` (Stop hook, L2-L4) é UA do Registro (carrega `mc-lint.rua.yml`) mas é peça pré-existente, irmã do RUA — referenciada, não parte deste artefato.

10 · Glossário técnico

Termo	Definição
UA	Unidade Agêntica — skill, agent, hook ou workflow.

Termo	Definição
Bloco rua:	Frontmatter de governança que cada UA carrega; fonte da verdade do Registro.
Fail-closed	Campo ★ ausente ⇒ UA fora do Registro canônico, visível como dívida.
Conjunto fechado	As 9 chaves que uma linha de auditoria pode conter; o firewall LGPD em forma de dados.
Sentinela	Linha de log com event iniciando em (ex.: rótulo de fixture); ignorada na métrica.
Critério de morte	UA que nunca rodou em 30d → candidata a rasgar (MC-SPEC §6).
Determinístico	Mesma entrada + mesmo commit → saída byte-idêntica.
ASE / L0-L6	Agentic Systems Engineering; camadas da stack. RUA cobre L1, L5, L6.

Hierarquia: Dignidade > Compliance > Viabilidade · **Proof-First** · **Zona Verde** (zero PII) · gerado, não mantido · **#50.2 Cabeça Única Status:** v1.0 · derivado · NÃO selado · NÃO no Vault · aferido contra o código real "É eu, tu, a Anciã e a esteira. ∞"